Magic2: Supplementary Technical Reference

Architecture, Algorithms, and Implementation

Jean Thierry-Mieg Danielle Thierry-Mieg Greg Boratyn

April 2026 — working draft

Abstract

Magic2 is a high-throughput aligner for RNA-seq and DNA-seq reads, supporting short reads (Illumina) and long reads (Nanopore, PacBio), developed at the National Library of Medicine, NIH. This document is a technical reference describing the algorithms, data structures, parallel execution model, and output formats that constitute the Magic2 implementation. It is intended as supplementary material to the main benchmarking paper, which reports alignment accuracy and throughput comparisons against HISAT2, STAR, and minimap2; readers interested primarily in results should consult that paper; those wishing to apply Magic2 should consult the user guide. Developers and advanced users wishing to understand, extend, or port Magic2 will find the full design rationale here.

Contents

1 Overview and design philosophy

Magic2 is a complete rewrite and extension of the earlier Magic-BLAST aligner (**?**), developed at the National Library of Medicine (NLM), NIH, itself building on the AceView gene and transcript annotation system (**?**). It aligns short (Illumina) and long (Nanopore, PacBio) RNA-seq or DNA-seq reads against an annotated or unannotated reference genome, augmented with its mitochondria, exemplars of its ribosomal RNA precursor and of its main repeat elements, and optionally its most frequent contaminants (Section 5.1). In a single pass Magic2 produces alignments, gene-expression counts, intron-support tables, poly(A) addition sites, candidate transcription start sites, coverage tracks, and rich QC metrics.

A posteriori, the name Magic2 can be parsed as *Memory-Aware Genomic Integrative Computation* — an acronym that reflects the central role of memory efficiency in the design.

Three principles govern all architectural decisions:

1. **Cache efficiency.** The dominant cost in large-scale genomic computations is data movements, not arithmetics. All core algorithms are designed to generate long sequential memory-access patterns rather than random accesses, keeping working data inside CPU caches and saturating memory bandwidth.

2. **Automatic configuration.** Read length, paired-end geometry, adaptor sequences, strandedness, and sequencing platform are inferred from the data. Internal parameters (seed length, maximum intron size, subsampling density) are derived from genome size and reported to the user for transparency. Manual overrides remain possible but are rarely necessary.

3. **Single-pass richness.** All integrative outputs — alignments, expression counts, splice-junction tables, coverage tracks, poly(A) calls, QC metrics — are produced during the alignment pass while reads and seeds reside in memory, avoiding the need for secondary tools.

The source code is available at `https://github.com/NLM-DIR/Magic2` (branch `master`). The main entry point is `wsa/sa.main.c`; the central header defining all structures and function prototypes is `wsa/sa.h`.

This document is a technical reference covering algorithms, data structures, parallel execution, and output formats. Benchmark results comparing Magic2 to current aligners are reported in the accompanying main paper; a separate user guide covers installation, index construction, and everyday usage.

2 Hardware constraints and the sort–merge paradigm

2.1 Memory hierarchy and its consequences

Modern CPUs execute each elementary operation in approximately one nanosecond when the operands reside in L1 or L2 cache, but fetching the same data from main RAM typically costs 60–100 ns on current x86 processors. A page fault to disk costs milliseconds. Because large genomic datasets far exceed cache capacity, algorithm design must prioritise access patterns over theoretical operation counts.

The core challenge in alignment is *pre-assignment*: given a short read, rapidly identify the small number of genomic loci where alignment is likely. Hash tables offer $O(1)$ average lookup, but on a multi-gigabyte genome index each lookup jumps to a quasi-random memory address, triggering cache misses and page faults. Measured latency per seed commonly reaches 100–300 ns, turning a theoretically fast algorithm into a memory-bandwidth bottleneck.

2.2 The sort–merge alternative

Magic2 adopts a *sort–merge* paradigm:

1. Generate seed records for reads and genome as large, contiguous blocks written sequentially to memory.
2. Sort both lists by seed value using a hard-coded cache-efficient radix sort (Section 7).
3. Join the two sorted lists by simultaneous linear traversal: both pointers advance monotonically, so the entire join operates strongly favors CPU cache and pre-fetching.

This approach is not novel in principle; it underlies high-performance database engines and has been applied in bioinformatics, notably in the DIAMOND protein aligner (**?**). Its roots lie in classical external sorting and merge-join algorithms (**?**), with modern refinements in cache-oblivious algorithms (**?**).

The key trade-off is replacing $O(n)$ cache hostile random hash look ups by two cache friendly operations: $O(n \log n)$ sorting followed by $O(n)$ sequential merging. The merge step is so cache-friendly that it runs at near-memory-bandwidth speed, far outpacing hash-table approaches on large genomes.

If avalailable, the whole preassignment is automatically offloaded to a GPU (Section 6).

3 Parallel execution: agents and channels

3.1 Channel library

Magic2 exploits multiple cores through a clean, lock-free framework in which independent worker threads (*agents*) communicate solely through *channels* — bounded, self-synchronising queues implemented in `wchannels/channel.c`, closely following the paradigm of Go channels.

A channel is typed and has a fixed capacity. An agent inserting into a full channel blocks until space is available; an agent extracting from an empty channel blocks until data arrives. This built-in backpressure eliminates explicit locks and condition variables within agents, greatly simplifying both implementation and reasoning about correctness.

Channels track the number of active producers. When all producers have finished and decremented the source count to zero, the channel closes automatically; consumers drain any remaining records and then exit cleanly. This guarantee holds regardless of the number of agents or the order in which blocks are processed: no data are lost, and termination is deadlock-free.

3.2 Pipeline topology

The master thread enqueues descriptors of independent data blocks containing up to `bMax` Mega-bases (default 3 Mb) into a shared input channel. A pool of agents retrieves blocks from this channel, executes the complete per-block pipeline (seed identification, alignment extension, adaptor detection, splicing, scoring mismatches, pairing), and forwards results to designated output channels. Dedicated consumer agents drain output channels to merge the per-chromosome wiggle profiles, cumulate the gene expression counts, the introns, the poly-A sites and compile QC tables.

Each agent follows a straightforward loop:

```
while ((block = channelGet(inputChannel)) != NULL) {
   processBlock(block);
   channelPut(outputChannel, result);
}
signalDone(outputChannel);
```

Agents never wait on one another; all synchronisation emerges from channel blocking and closure. The main program defines the pipeline topology by creating channels, setting source counts, launching agents, and seeding the input channel but neither the main program nor the agents contain any explict calls to the low level unix semaphores and other synchronization tools.

3.3 NUMA-aware execution

During its initialisation, Magic2 analyses the harware and counts the number of CPUs, nodes and eventual GPU. Then Magic2 re-spawns itself and binds to the least-loaded node, and eventually the least buzy GPU, minimising cross-node memory traffic. Users do not need set a thread count; Magic2 manages all concurrency automatically and continuosuly monitors its usage of memory by varying the number of datablocks processed in parallel.

For debugging, `--debug` forces fully serial execution (`nAgents=1`, `nBlocks=1`) with verbose output. `--numactl` suppresses re-spawning while retaining full parallelism.

4 Pilot-block inference and run-time adaptation

4.1 The problem: alignment parameters depend on the data

Several alignment decisions that strongly affect accuracy, cannot be made before reading the data: whether the library is RNA or DNA, what is the sequencing technology (Illumina, PacBio, Nanopore ...), are the reads long or short, single ends or paired, what are the absolute and relative rates or substitutions and indels, which is the dominant sequencing strand, and what adaptor sequences were ligated. Requiring the user to supply these parameters manually is error-prone and unnecessary, because the data itself carries all the evidence needed to infer them.

4.2 Pilot blocks

As the input file is parsed, the read-parser emits a continuous stream of data blocks (*BB*), each carrying up to `bMax` megabases (default 3 Mb). The first five blocks are designated *pilot blocks*. They enter the agent pool and are aligned immediately, before any global parameters are frozen.

On completion of each pilot block, Magic2 tallies the evidence accumulated so far:

- **Read type.** Long reads with relativelly frequent indels indicate Nanopore or PacBio; short, low-indel reads indicate Illumina.
- **Library type.** A significant fraction of reads spanning GT-AG or CT-AC boundaries identifies the library as RNA; the absence of such signals indicates DNA.
- **Strand.** Many RNA-seq protocols are strand-specific: 99 % of reads may align consistently to one mRNA strand. The ratio of GT-AG (sense) to CT-AC (antisense) intron signals and the orientation of the mitochondrial reads measures this directly and sets the strand bias used in all subsequent interpretation of the alignments.
- **Adaptor sequences.** Unaligned 3′ overhangs are accumulated into per-position base-frequency histograms (Section 9). Once the histogram is sufficiently populated the adaptor consensus is called.

Once enough pilot blocks have provided good evidence — in practice one block of 3 Mb is usually sufficient for a high quality run — all inferred parameters are frozen and used by all agents.

4.3 Reprocessing and zero read loss

If a pilot block was aligned before a parameter was finalised (for example, before the adaptors were known), it is silently reprocessed from scratch with the final parameters before its results are committed. All subsequent blocks, including the remaining pilot blocks still in the channel, proceed directly with the final parameters. No reads are discarded or excluded from the output: the pilot phase is entirely transparent to the user.

4.4 Benefit

The pilot mechanism means that a single undecorated command such as

```
magic2 -i SRR122334 -x IDX -o output
```

produces fully optimised alignments regardless of library preparation, sequencing platform, or adaptor chemistry. For example, strand bias is exploited globally: a 99 % strand-specific RNA library is handled exactly as if the user had explicitly declared the strand on the command line.

5 Seed generation and genome indexing

5.1 Completing the target

An aligner cannot report that a read originated from a sequence it was never given. It reports the best match among the sequences it *was* given. A read whose true source is missing from the target does not disappear: it lands on whatever else in the target resembles it most. Missing sequence therefore does not produce missing data, it produces false data — and the error is systematic rather than random, because the same reads land on the same wrong loci in every run.

This is the reason Magic2 is given a target *configuration* (`-T`) rather than a genome (`-t`). The genome is only the largest component of a target. The others are small, but they absorb reads that would otherwise be misattributed to it — either because their source is genuinely absent from the genome, or, as with ribosomal RNA, because their source is present in the genome yet cannot be indexed.

Code	Class	Content
`G`	Genome	Chromosome sequence
`M`	Mitochondria	Mitochondrial genome
`C`	Chloroplast	Chloroplast genome (plants)
`R`	Ribosomal	One or a few rRNA precursor sequences
`S`	Repeats	Exemplars of the main interspersed repeat families (Alu and other SINEs, . . .)
`E`	Controls	ERCC or other spike-ins
`B`	Bacteria	Expected bacterial contaminants
`V`	Virus	Expected viral contaminants
`A`	Annotation	GTF/GFF, or a bare intron list

Ribosomal RNA. rRNA is the extreme case, and it is extreme in a way that is easy to misdescribe. It can represent up to 80 % of a total-RNA library, and several percent survives even poly(A) selection or ribodepletion. But unlike the mitochondrion it is not a separate genome. Every rRNA molecule is transcribed from the nuclear chromosomes, from exactly one of the hundreds of near-identical rDNA repeats tandemly arrayed there. Class `R` therefore corresponds to no physical object. It is a bookkeeping construct, a stand-in for that whole collection of loci, adopted because the question it suppresses — which of the hundreds of copies emitted this particular molecule — is at once unanswerable and uninteresting. No experiment asks it.

Every experiment asks how much rRNA the library contains, which is a property of the collection and not of any member of it.

The technical difficulty is that the true sequence, though present in the assembly, is invisible to the index. Hundreds of tandem copies place the rDNA seeds far above `maxTargetRepeats`, so the multiplicity filter of Section 5.6 discards precisely them, while the rRNA-derived pseudogenes and degenerate fragments dispersed across the genome are each rare enough to survive it. The filter does not fail to see the repeats: it removes them by design, and in doing so hands the reads to the only rRNA-like sequence left standing, which is the wrong one. The result is not missing data but misplaced data — a haze of spurious expression and spurious junctions over loci that merely resemble rRNA. The benchmark in the main paper attributes the elevated novel-junction counts of competing tools on total-RNA libraries to this effect.

Declaring one or a few precursor sequences as class `R` restores an indexable copy. Several non-identical precursors may be supplied, since the arrays are not homogeneous. This works only because seed multiplicity is counted *per class* rather than across the target as a whole. The classes are thus not merely accounting labels. Were the count global, the exemplar's seeds would be masked along with the genomic rDNA they duplicate, and the construct would defeat itself. Counted within class `R` alone they occur once or twice, pass the filter untouched, and give the reads somewhere correct to go.

A tested alternative: masking. If the rDNA seeds in class `G` are the problem, the obvious move is to delete them: during target construction, mask from `G` every seed also present in `R`. This is easy to implement, and it was tried. It measurably reduced the number of perfect alignments, and was abandoned. The reason is that rRNA-like sequence in the genome is not all parasitic; some of it is genuinely transcribed or genuinely sequenced, and deleting its seeds removes true alignments along with false ones. The classes are therefore left deliberately overlapping, and the competition between them is settled by score rather than by exclusion.

Interspersed repeats. Class `S` is the same construct as `R`, applied to the main interspersed repeat families — Alu and the other SINEs, and their kin. The situation is formally identical: Alu occurs about a million times in the human genome, so far above `maxTargetRepeats` that its seeds cannot survive in class `G`, and a read lying wholly inside an Alu cannot in any case be assigned to one copy rather than another. Declaring exemplars as class `S` lets such reads be captured and counted globally, as a measured property of the library, rather than scattered or lost. Reads that extend into the unique sequence flanking a repeat are unaffected, and continue to be placed at their true locus by class `G`.

Mitochondria. The mitochondrial genome is a circular molecule of 16.6 kb, against 3.1 Gb of nuclear sequence, but it is present in hundreds to thousands of copies per cell, and its transcripts routinely account for 5–30 % of an RNA-seq library. It is usually already present in the genome FASTA, so declaring it separately is not a question of presence but of accounting and of competition. Every mammalian genome carries *NUMTs*, nuclear insertions of mitochondrial sequence; when the true mitochondrion is not an explicit, exact target, mitochondrial reads settle on NUMTs and manufacture nuclear expression where there is none.

Class `M` also earns its place functionally. Mitochondrial transcription is intronless, polycistronic, strand-defined and very highly expressed, which makes it a dense and unambiguous strand calibrator, independent of splice signals. The pilot phase uses it as such (Section 4): the orientation of mitochondrial reads settles the strandedness of a library even when introns are scarce or absent.

The class bonus. Residency in the index is necessary but not sufficient. A real rRNA read still competes against hundreds of degenerate genomic copies; it carries sequencing errors, and the individual's rDNA differs from the exemplar by polymorphism. The comparison is thus not between one true match and one false one, but between a single true template and the *best of hundreds* of false ones — and the maximum over many

near-misses will, by chance alone, sometimes exceed the truth. This is a multiple-comparisons effect: it biases assignment away from the correct class systematically, and the more numerous the decoys the worse it gets. Mitochondria against NUMTs pose the identical problem.

Magic2 corrects it with a fixed bonus: alignments to classes `M`, `C` and `R` score $2 \times$ `errCost` higher than they otherwise would, so an organellar or ribosomal placement prevails unless a genomic placement is better by more than two substitutions. Expressing the bonus in error units rather than absolute points keeps it calibrated when `errCost` switches between 4 and 8 on the pilot data (Section 4). Classes `E`, `B` and `V` receive no bonus and need none: spike-ins, bacteria and viruses are alien to the genome and have no imitations there to compete with. The bonus exists exactly where a class competes with genomic look-alikes of itself, and nowhere else.

Three mechanisms thus act together, and each is needed. Per-class multiplicity counting keeps the true sequence in the index. The same per-class count is what decorates each seed record, so during clustering (Section 7.7) an `M` or `R` glue point keeps its full weight while its genomic imitations are down-weighted for high multiplicity. And the bonus settles the residual competition on score.

Controls and contaminants. ERCC spike-ins (class `E`) enter the library at known concentrations, and so calibrate sensitivity and dynamic range against an absolute scale. Expected bacterial and viral sequence (classes `B`, `V`) follows the same logic as rRNA: declared, it is quantified; undeclared, it is misplaced.

Cost. The whole completion — rRNA exemplars, mitochondrion, spike-ins, common contaminants — amounts to a few hundred kilobases against a 3.1 Gb genome, and so is free in index size, index construction time and alignment time. It changes the disposition of a large fraction of the reads. Few interventions in the pipeline have so asymmetric a payoff, which is why the annotated, completed target is the recommended configuration and the bare genome (`-t`) is offered only for testing.

5.2 Seed encoding

Magic2 uses exact k-mer seeds of length $k \leq 19$, encoded as 32-bit unsigned integers (2 bits per base, so 16 bases fill exactly 32 bits). The default k is chosen automatically from genome size:

- $k = 16$ for small genomes below 1 gigabases, e.g. *C. elegans*, *D. melanogaster*);
- $k = 18$ for large genomes (human, mouse).

When $k > 16$, the extra bases are handled by radix partitioning into 16 or 32 separate seed tables indexed by the least significant base pair(s), providing effective 18- or 19-mer specificity while keeping each individual table within 32 bits.

Continuous (ungapped) k-mers are preferred over spaced seeds: spaced seeds are counterproductive for indel-prone technologies (Nanopore, PacBio) because indels in the spacers disrupt the spaced-seeds.

5.3 Strand-agnostic seed generation

To halve the index size and enable strand-agnostic matching, both the forward k-mer S and its reverse complement C are computed simultaneously using an incremental rolling update:

$$S_{\text{new}} = \big((S \ll 2) \mid b\big) \;\&\; \text{mask} \tag{1}$$

$$C_{\text{new}} = \big((C \gg 2) \mid (b' \ll (k-2))\big) \;\&\; \text{mask} \tag{2}$$

where b is the next base (2-bit coded) and b' its complement, and $<< 2$ or $>> 2$ means that the k-mer is shifted left or right by 2 bits before insertion of the new base. The strand-agnostic minimal value $Z = \min(S, C)$

is retained. The coordinate is encoded as $(x \ll 1)$ if S was chosen (forward strand) or $(x \ll 1 \mid 1)$ if C was chosen (reverse strand). So x is even if the chosen seed Z is extracted from the plus strand and odd if extracted from the minus strand. Ambiguous bases reset the rolling accumulator; a seed is emitted only after k consecutive unambiguous bases.

5.4 Paired-read encoding

Paired-end reads arrive with names of the form `xx/1` and `xx/2`. Only the base name `xx` is entered into the read dictionary (`dictAdd(dict, "xx", &n)`), yielding a compact integer n. Then mate 1 and mate 2 are stored as contiguous numbers $2n$ and $(2n + 1)$ such that sorting by reads automatically implies sorting by pairs. The alignment stage therefore receives tables in which read-pair processing requires no further regrouping.

5.5 Subsampling

To reduce index size without sacrificing alignment sensitivity, seeds are sub-sampled using a *rolling-min-hash*: a round-robin buffer of size p (default 6) stores the hash of Z at recent positions; only the position with the minimum hash among the last p values is retained as a seed. This guarantees:

- average inter-seed spacing of $p/2$ positions;
- maximum gap of p positions;
- near-perfect synchronisation between read and genome seeds (in practice alignment begins by the second seed even when $p = 6$).

5.6 Target multiplicity filtering and jump encoding

After collection, target seeds are sorted (Section 7) and their multiplicity (number of occurrences in per target) is counted. Seeds with multiplicity $> Q$ (default 81) are discarded to limit spurious hits from repetitive elements; empirical tests showed $Q = 31$ gives slightly higher specificity while $Q = 81$ slightly reduces false negatives at acceptable cost in index size and runtime.

Jump information is then embedded directly in the seed records: every 256th seed stores pointers to future seeds at distances $n_1 \times 256$, $n_2 \times 256$, $n_3 \times 256$, $n_4 \times 256$, enabling fast skip-ahead during the merge step.

The filtered, jump-augmented taget table is computed only once during index preparation and written to disk in compact binary format. It is later memory-mapped in shared read-only mode when aligning sequence data.

5.7 Junction seeds from annotated splice sites

When a GFF/GTF annotation is supplied, Magic2 supplements genomic seeds with *junction seeds* spanning annotated exon–exon boundaries, improving detection of short exons with known junctions.

For each annotated mRNA, all exons are concatenated *in silico* to form a continuous spliced transcript. All k-mers straddling a junction with at least 4 bases on each side ($k - 4$ starting positions per junction, with $p = 1$ so every valid position is used) are generated. For each such k-mer, two seed records with identical value Z are emitted:

- one pointing to the donor site (end of upstream exon);
- one pointing to the acceptor site (start of downstream exon), augmented with ancillary phase-shift information encoding the exact junction offset.

This dual-record strategy ensures that a single seed match overlapping a junction can trigger two extensions, one on each side of the splice site.

The additional index size is negligible: with $k = 18$ and ~300,000 annotated human introns, $(k-4) \times 300{,}000$ yields only a few million additional seeds, compared to around half a billion genomic seeds. Junction seeds are merged with the genomic table, sorted, and stored together in the same binary files.

6 Sorting, joining and optional GPU acceleration

6.1 Data structures

Seed records are `CW` structs (16 bytes, 16-byte aligned),

Field	Type	Meaning
`seed`	`unsigned int`	32-bit 16-mer seed
`nam`	`unsigned int`	sequence identifier and strand
`pos`	`unsigned int`	coordinate of first base of seed
`flags`	`unsigned int`	auxiliary information

The 16-byte size is deliberate: four `CW` records fill one 64-byte cache line exactly, which is optimal for both the CPU vectorized instructions and for GPU coalesced memory access.

7 Radix sort

Radix sort runs in $O(n)$ time and generates purely sequential memory accesses, making it strictly preferable to comparison-based sorts once the table exceeds cache capacity.

Magic2's implementation (`wsa/sa.sort.c`) sorts the 32-bit `seed` field of 16-byte `CW` records in three passes with bit-widths of 11, 11, and 10. All three frequency histograms are accumulated in a single linear scan before any data movement, so the table is read sequentially exactly four times in total. The scatter passes ping-pong between the source array and a 128-byte-aligned scratch buffer; the sorted result lands in the scratch buffer with no final copy. On SSE2-capable processors each 16-byte record is moved by a single `_mm_load/store_si128` instruction, with a portable `memcpy` fallback elsewhere.

7.1 Data loading and joining to the read-seeds with the target-seeds

The filtered multi-target index (Section 5.2) is partitioned into N sub-tables (16 or 32, indexed by the least-significant base pair(s) of the seed value) and all partitions are memory mapped. at the start of the program.

The DNA or RNA sequences to be analyzed are read sequentially, or automatically streamed from the NCBI/SRA archives, and partitioned in blocks of approximately $bMax$ megabases (default 3Mb). As soon as a block is complete, its processing starts, and the block is discarded once it has been fully exploited. And since the number of active block is controlled by channelling back-pressure (Section-3.1), an arbitrarilly long input file (say 1 Tb) can be analyzed on a 32 Gb machine. This also means that the latency when acquiring the fasta/fastq files is hidden behind the parallel processing of the already available data.

For each block, the read seeds are constructed exactly as the target seeds (Section 5.2), with the same pseudo-periodicity p or a divisor of p (section 5.5), then sorted (Section 7). Both tables are scanned in parallel, advancing the pointer on the side of the lower seed. When there is a coincindence, the local Cartesian product

of all entries with the matching seed is emitted as 64 bytes SEEDMATCH structs, by simply joining the 2 matching records. This table is then sorted by read (hence automaticaly by pairs, section-5.4).

7.2 Optional GPU acceleration

When an NVIDIA GPU is available, Magic2 offloads three steps of the seed-matching pipeline to the device via CUDA Thrust (`wsa/sa.gpusort.cu`): sorting the read-seed table, and joining it against the pre-loaded genome index, then sorting the table of SEEDMATCH.

GPU use is activated at compile time (`-DUSEGPU`) and detected at run time; execution falls back silently to the CPU radix sort (Section 7) when no compatible device is present. If a GPU is present, the genome seed-index is transfered to the GPU prior to the calculation of the first data block and discarded from the CPU side. All structure declarations are shared and since both the CPU and the GPU use 64-bytes aligned buffers, no data conversion is required at the C/C++ CPU/GPU boundary.

7.3 Latency hiding through channel-based concurrency

The GPU call in each agent is synchronous: the agent blocks on the device until the sorted `SEEDMATCH` table is ready, then resumes. No asynchronous CUDA streams, double-buffering, or explicit staging logic are required.

The standard GPU programming approach to this problem uses CUDA streams with asynchronous transfers (`cudaMemcpyAsync`): while the device processes block N, the host prepares and begins transferring block $N + 1$ into a second buffer, so that the PCIe transfer latency of one block is overlapped with the GPU computation of the next. This *double-buffering* pattern is effective but requires the programmer to manage two buffer sets, carefully sequence stream operations, and reason explicitly about which synchronisation points are safe — a non-trivial engineering burden that is easy to get wrong silently.

Magic2 avoids this complexity entirely. The agent pool, coordinated by bounded blocking channels, provides CPU–GPU overlap as a natural consequence of its concurrency model. While one agent is blocked waiting for the GPU, all other agents continue executing their own blocks on CPU — parsing fastq files, extending alignments, writing coverage tracks, and accumulating junction and QC tables. The blocked agent simply parks at what is, from the pipeline's perspective, an ordinary blocking-channel operation, indistinguishable from blocking on an empty input queue, or on loading a file from a disk. Provided the number of active agents is sufficient to keep both the GPU and the CPU cores occupied — a condition that holds automatically given Magic2's block-size and agent-count defaults — GPU latency is fully overlapped with productive CPU work on other blocks, with no GPU-specific orchestration code in the pipeline at all. After a while, the different agents spontaneously anti-synchronize and relativelly rarely require the GPU at the same time.

The channel library (`wchannels/channel.c`) is a direct C implementation of Go's channel semantics (**?**), themselves rooted in Hoare's Communicating Sequential Processes (**?**). It was written to bring Go's concurrency model to an existing 700,000-line C codebase that could not be migrated to Go. Go programmers obtain the same GPU-latency-hiding property for free from the goroutine scheduler, which parks a goroutine blocked on a channel and runs another in its place. Magic2's agent/channel architecture replicates this behaviour in C, without any GPU-specific orchestration code in the pipeline.

7.4 Per-block GPU pipeline

GPU acceleration is applied at the block level: each agent, upon receiving a block, offloads the seed-matching phase to the device, then resumes CPU-side alignment extension once the device returns. The genome seed index is resident in device memory throughout the run, having been transferred once before the first block is

processed; the CPU no longer needs to memory-map the target index, about 22 GB for the human genome, and can discard it.

Within each block the GPU performs the following steps in sequence:

1. **Read-seed sort.** The read seeds extracted from the current block are sorted on the device by seed value, producing the same ordered table that the CPU radix sort would deliver (Section 7).

2. **Common-seed identification.** The sorted read-seed table is intersected with the resident genome index to find all seed values present in both. For each common value the set of matching genome positions and the set of matching read positions are identified.

3. **Match-record emission.** For each common seed with m genomic occurrences and n read occurrences, all $m \times n$ candidate hit pairs are written to a match table in parallel, using a prefix-scan to assign output positions without atomic operations.

4. **Sort by read pair.** The match table is sorted by read identifier. Because of the paired-end encoding (Section 5.4), this simultaneously groups hits by read pair and places mate 1 immediately before mate 2, delivering a table directly consumable by the CPU alignment stage with no further reordering.

5. **Return to CPU.** The sorted match table is copied back to host memory. All subsequent processing — alignment extension, splicing, scoring — proceeds on the CPU without any further access to the genome index.

7.5 LibTorch backend: an alternative GPU implementation

The pipeline of Section 7.4 is also implemented a second time, independently, on top of LibTorch, the C++ API of PyTorch (`wsa/sa_torch.cpp`, interface `wsa/sa.torch.h`). It is compiled under `-D USE_TORCH` into the binary `magic2_torch`, and is selected by the same respawn mechanism. The two backends are alternatives, not layers: each performs the complete pre-assignment, and either can be replaced by the CPU sort–merge of Section 7 at run time.

The motivation is expressive rather than numerical. The entire join is written as a handful of whole-array operations — `bucketize`, `index_select`, `repeat_interleave`, `argsort` — with no hand-written kernel, no explicit thread or block geometry, and no manual memory management. The same source runs on CUDA, on Apple MPS, and on the CPU, the device being selected at run time. Isolation follows the rule already used for the SRA reader: the file includes only LibTorch headers and its own public header, never `sa.h` or any acedb header, and all data crosses the boundary as flat pointers and counts behind an opaque `SATorchObj` handle.

Index residency. Each of the $NN = 16$ partitions is held on the device as three tensors in compressed-sparse-row form: `g_keys` $[K]$, the sorted unique seeds; `g_offsets` $[K+1]$; and `g_cw` $[M, 3]$, the payload columns `nam`, `pos`, `flags`. The seed column of the `CW` record is not stored: the CSR index already says which rows belong to which seed, and the value itself is never needed after the join. Dropping it reduces the resident index for a human or mouse genome (1.43 billion records) from about 22.8 to 17.1 GB, which leaves room on a 32 GB device for the 1–2 GB of working tensors a block requires. Seeds are compared as *signed* `int32`, LibTorch offering no unsigned 32-bit `bucketize`; the on-disk index must therefore be built with the matching signed radix sort.

Join and output. Read seeds are located in `g_keys` by `bucketize`, equality is verified, and the CSR start and count of each surviving seed are gathered. `repeat_interleave` then materialises the Cartesian product directly, the jump lines of Section 5.6 being excluded by masking the row indices divisible by 256, so no extra field is needed. The genome payload is gathered, the five output columns are concatenated, and the table is sorted by read identifier — which, by the paired encoding of Section 5.4, groups mates as before. The record returned is a `TORCHMATCH`: five 32-bit fields, 20 bytes, against 64 for a `SEEDMATCH`, the two seed fields and the read flags having been dropped as always zero. The buffer is owned by the handle and loaned to the C caller. Two entry points are offered: `saTorchJoinSort`, which joins and sorts seeds already extracted by the CPU, and `saTorchCodeJoinSort`, which in addition extracts the seeds on the device from a flat IUPAC buffer, packing the forward and reverse-complement k-mers by weighted sums and keeping $Z = \min(S, C)$ exactly as in Section 5.3. Every entry point is exception-guarded and returns a null pointer on any failure, which the caller reads as an instruction to fall back to the CPU.

Performance and its ceiling. Both GPU backends are correct and in production, and both are only marginally faster than the CPU sort–merge. This is a property of the problem rather than a defect of either implementation, and it can be measured directly. Varying the sampling steps changes the size of the seed tables while leaving every other stage of the pipeline untouched, and therefore isolates the cost of pre-assignment. Across 27 benchmark datasets, tripling the number of read seeds (read step 3 to 1) costs only 10 % of wall time, and doubling the number of genome seeds (index step 6 to 3) costs 22 %. Back-solving each against $T = (1 - f) + kf$ gives a read-seed-dependent fraction $f \approx 5\,\%$ and a genome-seed-dependent fraction $f \approx 22\,\%$. Seed generation, sorting and joining — the entire work that either GPU backend performs — thus accounts for roughly a quarter of the run. By Amdahl's argument, no implementation of this stage, on any device and however fast, can accelerate Magic2 by more than about $1/(1 - 0.27) \approx 1.4\times$. The observed marginal gain is exactly what that bound predicts, and it is the reason no further GPU implementation of the seed stage is planned.

The remaining three quarters — alignment extension, the exon-path dynamic programming, and the splice logic (Sections 7.5–8) — are branch-heavy, data-dependent and irregular, which is the least favourable workload a GPU can be given; they stay on the CPU by design. The clearer benefit of both GPU paths therefore lies elsewhere: the index is resident on the device, so the CPU no longer memory-maps ∼22 GB per machine and that memory is returned to the agents. Given the cost of a GPU, the CPU path remains the recommended default.

The alignment extension stage receives `SEEDMATCH` records — pairs of (genome coordinate, read coordinate) for matching k-mers ($k \approx 18$), sorted by read pair. Two flavours exist: standard genomic seeds (Section 5.2), and junction seeds which each produce two glue points (a donor and an acceptor on either side of an annotated intron boundary, section 5.7).

7.6 Strand resolution

Seeds are strand-agnostic: chromosome and read identifiers encode strand implicitly — even identifiers $2n$) indicate the plus strand and odd identifiers $(2n + 1)$ indicate the minus strand (section 5.3). The exclusive-or bitwise operation `(chrom ^ read) & 1` therefore determines whether the read and the chromosome at the position indicated by their common seed are locally parallel or anti-parallel. This flag is extracted and dropped; from this point all processing works only on parallel alingments by compairing the positive-strand of the read (`dnaR`) to either the original target sequence (`dnaG`) or to the target reverse-complemented sequence (`dnaG_RC`), both memory-mapped at initialisation.

7.7 Clustering and locus selection

For each read, glue points may scatter across many chromosomes but cluster into one or a few genuine loci. Loci are scored by glue-point counts, with seeds of high genomic multiplicity (recorded in the `SEEDMATCH` record, section 5.6).) down-weighted. Junction seeds are tallied separately and contribute additional evidence for a specific splice structure. Within each candidate locus, glue points are sorted by chromosome, strand, then by the diagonal value $a - x$ (genome coordinate minus read coordinate); in the absence of indels all seeds from the same exon share the same diagonal.

7.8 Seed extension: the jumper algorithm

Starting from each seed match, Magic2 extends the alignment using the *jumper* algorithm, first described in the AceView human-genome aligner (**?**) and subsequently used in Magic-BLAST (**?**), to which we refer for a full description.

Extension proceeds base by base: if `*cp & *cq != 0` (a bitwise test that handles IUPAC ambiguity codes: R matches A or C, and N matches any letter), both the read pointer `cp` and the genome pointer `cq` advance by 1 base. On a mismatch, a *jumper table* is consulted. Each row encodes one error hypothesis as a triple `(dx, da, n)`: advance the read pointer by `dx` and the genome pointer by `da`, if the following n letters check as exact matches, otherwise try the next hypothesis. The table covers substitutions, insertions, and deletions of up to 10 bases (Table 1); rows with $dx + da > k$ can be ignores, in many places Magic2 uses the limit $k = 3$; a sentinel entry `{1, 1, 0}` with $n = 0$ is always accepted and terminates the search without a special exit condition. Extension halts optionally on the first mismatch, or when more than 5 errors are detected over less than 10 bases, or when the sequence ends.

Table 1: Jumper table (representative rows)

dx	da	check	meaning
1	1	4	substitution
1	0	4	insertion in read (1 base)
0	1	4	deletion in read (1 base)
2	0	6	insertion (2 bases)
0	2	6	deletion (2 bases)
⋮	⋮	⋮	⋮
10	0	12	insertion (10 bases)
0	10	12	deletion (10 bases)
1	1	0	sentinel (always accepted)

The jumper algorithm is strictly linear and cache-friendly, and can drift freely along any diagonal — unlike banded Smith–Waterman, which is constrained to a fixed band and cannot follow a read that accumulates locally clustered errors. Smith–Waterman is optimal only under the assumption that sequencing errors are independently distributed, a hypothesis that does not hold in practice (polymerase slippage, homopolymer runs, and cycle-dependent quality decay are all correlated).

Another advantage is that one can define several jumper tables and select dynamically the most appropriate on a per-run or even per-read basis, or try several choices. When the observed number of indels is high, or if deletions are favored over insertions, or the contrary, Magic2 can switch tables and can adapt the maximal acceptable value for $da + dx$.

7.9 Junction-seed inward extension

Junction-seed glue points are extended inward toward the exon body: the donor glue point (at the $3'$ end of the upstream exon) is extended $3'\to5'$, and the acceptor glue point (at the $5'$ end of the downstream exon) is extended $5'\to3'$. Together they define a candidate alignment split across the intron without ever examining the intronic sequence, which is absent from the read. A seed hit is considered redundant if it falls within ±3 diagonals of an already-extended fragment; hits further off-diagonal are extended independently to accommodate small indels.

7.10 Dynamic programming — best exon path

Input. The input to the DP is a list of candidate exon fragments for a single read, each carrying: read and chromosome identifiers, genomic and read-space coordinates, and the error table produced by the jumper algorithm (Section 7.8). A fragment may consist of several exon if a junction seed has been unambiguously recognized. Fragments are sorted by their $5'$ genomic coordinate before the DP begins.

DAG construction and path extension. The DP maintains a list of active paths, initially empty. It scans candidate exons from $5'$ to $3'$ and at each step attempts to append the current exon to every existing path whose topology is compatible: the new exon must lie downstream on the same chromosome and strand, with a genomic gap consistent with an intron or with direct adjacency. When a compatible path is found, the exon is appended and the path score is updated as the running sum of individual exon scores, ignoring overlaps in read coordinates for now (those are resolved later by the adjust-introns step, Section 8). A new single-exon path is also started at each exon, so that paths beginning at any position are considered.

Pruning — optimality in a DAG. The candidate set naturally forms a directed acyclic graph (DAG): multiple paths can converge on the same exon from different upstream histories. When two paths can both be extended by the same next exon, they represent two routes between a common start and a common continuation point. The path with the lower running score is pruned at this point. This pruning is optimal: because all future extensions apply identically to both paths, the lower-scoring path can never overtake the higher-scoring one regardless of what follows. Pruning keeps the active path list small and prevents combinatorial explosion on reads that cross regions with many overlapping fragment candidates.

Output and downstream refinement. At the end of the scan, the surviving paths are passed to the adjust-introns and adjust-exons steps (Sections 8–8.3), which resolve overlaps, optimise splice boundaries, realign against the genomic shadow, and compute final scores. Only then are paths ranked and the best retained. Some human genes have exact paralogues or occur in tandem repeats in the genome; reads from these loci will genuinely map to multiple locations with equal scores and are reported as multi-mapping alignments. This is expected and correct behaviour: the multi-mapping rate is a property of the genome annotation, not an aligner artefact.

8 Intron detection

After the dynamic-programming pass, the successive fragments have not yet been clipped, their read coordinates very often overlap by a few bases and their target coordinates leak into true intronic regions. This is unavoidable since there is at least one chance in four that the last base of a true exon matches the first base of the intron, and one in four that the first base of the next exon matches the last base of the intron. In both cases the two successive fragments will overlap by at least one base. To locate the most probable correct boundaries and clip blunt-end the successive fragments the program works as follows.

8.1 Error minimisation

Because the genome contains so many repetions at all scales, two successive fragments often overlap over dozens of bases or more. The code first searches the region where a blunt end clip would minimize the total number of remaing errors. In an ideal situation where the sequencing is perfectly accurate, there would be no mismatch in the exonic regions but possibly several where the fragments leak into the intronic region. In such a case, this minimisation eliminates all mismatches, leaving a shorter perfect overlap. Real situations may be more complex.

Splice-signal selection. If the run is recognized as RNA, or during the pilot phase (sections 4), the optimal overlap is scanned successively for an already annotated boundary, then for a splice signal:

1. GT-AG (canonical sense) or CT-AC (canonical antisense), chosen according to the strand established during the pilot phase;

2. GC-AG (less common but well-documented);

For non-stranded libraries both GT-AG and CT-AC are considered. In case of success, the fragments are clipped accordingly, even if this produces a single base mismatch, insertion or deletion, and the score of the aligment receives a bonus. However, no bonus is given if the read mixes sence (GT-AG) and antisense (CT-AC) signals, or the the signal is incompatible with the strand specificity of the run.

The final clip point therefore simultaneously minimises alignment errors and maximises splice-signal evidence.

Minimal intron size. Magic2 does not call introns below 30 bases. No confirmed human intron shorter than 30 bases is known, and short genomic gaps in an alignment are more parsimoniously explained by sequencing deletions: a 6-base gap is reported as two successive 3-base deletions and penalised accordingly in the score. Above 30 bases, a gap becomes an intron candidate.

Minimal intron support. Confidence in a candidate intron is graded rather than binary, and it is set by the support n: the number of independent reads whose alignment crosses that exact donor–acceptor pair. The data themselves calibrate the scale. Introns with high support are almost always canonical GT-AG, whereas the proportion of GT-AG falls as the support decreases, revealing the level at which the collection starts to be dominated by noise. Magic2 exploits this by a descending procedure. The support threshold t is walked downward from the maximum observed support through the low values $5, 4, 3, 2, 1$, and at each step the cumulative number of GT-AG introns with support $\geq t$ is plotted against the cumulative number of introns of any splice signal with support $\geq t$. The local slope of this curve, after regularisation, is the fraction of newly admitted introns that are canonical. The walk stops as soon as that slope drops below $1/2$, that is, as soon as lowering the threshold one step further admits more non-canonical than canonical introns; introns with support below the threshold reached are not reported. This adaptive procedure is the one introduced in the Magic-BLAST paper (**?**); Magic2 inherits and extends it.

Intron scoring bonus/penalty. Once a gap is confirmed as an intron, its contribution to the alignment score is adjusted by one additional `errCost` unit depending on the library type inferred during the pilot phase (Section 4). In RNA mode the intron receives a bonus of `+errCost`: a spliced alignment is rewarded relative to an unspliced one, favouring genuine gene models. In DNA mode the intron receives a penalty of `−errCost`: a genomic gap is treated with mild suspicion, which disfavours retroposon insertions and other spurious long-range matches that would otherwise masquerade as spliced alignments. The net effect is that the same scoring framework handles both RNA-seq and DNA-seq libraries correctly without any mode-specific

code paths. Two adjacent exon fragments often overlap in read coordinates: both extensions reached into the intron and claimed the same read bases. The overlap is resolved by choosing a clip point — a position in read space at which the left fragment ends and the right fragment begins — that minimises the total number of errors (substitutions plus indels of up to 3 bases) summed across both fragments. In low-complexity regions the overlap can span 30 or more bases with errors on both sides, so this joint minimisation is essential for accurate boundary placement.

8.2 Adjust-exons: shadow realignment

After intron boundaries are fixed, the path coordinates are used to extract the *genomic shadow* of the read: the exact genomic sequence spanned by the alignment, with introns removed. The full read is then realigned against this personalised reference using the jumper algorithm.

This second pass resolves two classes of residual error. First, a short gap in the read that was previously interpreted as a sequencing indel may, when viewed against the shadow, be recognisable as a mis-measured intron boundary; adding two to five missing genomic bases from the better-supported side can create a clean GT-AG signal that was invisible in the original alignment. Second, errors near exon boundaries are sometimes artefacts of the clip point rather than true mismatches; realignment against the shadow redistributes them correctly.

After shadow realignment, all mismatches and indels are re-projected into exonic read coordinates, and the final alignment score is computed (Section 8.3). This is the score used for all downstream comparisons.

8.3 Alignment scoring

The score of a single-read alignment path is:

$$s = A - \texttt{errCost} \times E$$

where A is the number of aligned bases in read coordinates (each read base counted once, regardless of how many exons it spans), E is the total number of errors (substitutions plus indels of 1–3 bases), and `errCost`=4 is the default penalty per error. Genomic gaps of 4–29 bases are treated as deletions and contribute to E (a 6-base gap counts as two successive 3-base deletions, penalised as $2 \times$ `errCost`). Gaps of 30 bases or more are candidates for introns and do not contribute to E; their status is confirmed by splice-signal evidence and support count as described in Section 8.

8.4 Pair scoring and multi-mapping resolution

For paired-end libraries, the two mates of a pair are scored jointly. If both mates align at the same locus with correct relative orientation and a plausible insert size, their individual scores are summed to form a *pair score*. The pair score is then compared across all candidate loci for that read pair.

Consider a pair that aligns at two loci (both mates present at each) and also has an orphan alignment of one mate at a third locus. The orphan is discarded: a validated pair score always dominates a single-mate score, because the joint alignment provides roughly twice the evidence. All loci whose pair score equals the best pair score are retained as equally supported placements; this set is reported as the multi-mapping output for that pair.

For single-end libraries there are no pair scores; the best individual score (or all tied-best scores) is retained.

Reads or pairs whose best score falls below `minScore`=30 or whose aligned length is below `minAli`=30 are discarded.

8.5 Long-read rafias

Nanopore and PacBio reads are long enough to span many exons in a single read, but their error rate is substantially higher than Illumina and their errors are not uniformly distributed: certain sequence motifs, homopolymer runs, and locally low-complexity regions can produce a stretch of 30 to several hundred bases that cannot be reliably matched to the genome. In such cases the dynamic-programming step may produce two separate paths for what is biologically a single continuous alignment: for example, exons 1–7 on the left and exons 9–14 on the right, with a gap in between that no seed covers.

If the two paths share a unique genomic locus, have consistent orientation, and the unmatched read region is shorter than the genomic gap between the two flanking exon groups, Magic2 merges them into a single structure called a *rafia*. The unmatched central portion is called the *bubble*. The bubble is assumed to correspond to a real genomic region between the flanking exons — perhaps a difficult-to-sequence segment or a region absent from the current genome assembly — but its precise internal coordinates are unknown.

Scoring. Bubble bases are not penalised as substitutions. The alignment score is computed from the matched flanking exons only, exactly as for a standard path (Section 8.3). This prevents a long unsequenceable stretch from artificially depressing the score of an otherwise high-quality alignment.

BAM output. No single standard CIGAR operator can represent a bubble: the read bases are present and real, but their precise genomic location is unknown within a large reference span. Magic2 encodes a rafia using a compound `I`/`N` pair placed at the bubble boundary:

```
700M200I3000N500M
```

where `700M` and `500M` stand for the left and right exon groups (each expanding to their own `M`/`I`/`D` operations in practice), `200I` records the 200 bubble bases as present in the read but not consumed from the reference, and `3000N` records the 3000-base reference span that is skipped. The adjacency of `200I3000N` signals that these two events are coupled: the 200 read bases are not a sequencing insertion but an unlocatable fragment sitting somewhere within the 3000-base genomic region. The full read sequence is preserved in the `SEQ` field across all bases, so scoring and downstream re-analysis are unaffected.

This encoding is syntactically valid under the SAM specification, which does not forbid adjacent `I` and `N` operators. Tools that compute read length as the sum of read-consuming operators will recover the correct length. Some tools that interpret `I` strictly as a short sequencing insertion may flag the record; this is a known limitation of the BAM format for long-read structural variants.

[TODO — implement and verify this CIGAR encoding for rafias in `wsa/sa.sam.c`, and test against samtools, IGV, and any downstream tools used in the Magic2 pipeline.]

Rafias are rarely encountered in Illumina data: short reads that hit an unreadable stretch typically fail to align altogether rather than producing two flanking paths.

9 Adaptor detection and trimming

Adaptor sequences may be supplied via `--adaptor1`/`--adaptor2` or via the optional third column of the run manifest (`adaptor1=atgc,adaptor2=tggt`; see the user manual), but this is not required. In practice, the exact adaptor sequence as it appears in sequenced reads is frequently unknown to the user, owing to the complexity of modern library constructions. Magic2 therefore infers adaptor sequences directly from the data.

9.1 Overhang accumulation

After alignment, the unaligned 3′ tail of a read — the overhang — is a candidate adaptor fragment. Magic2 accumulates four per-position base-frequency histograms over overhangs, one for each possible first overhang base (A, T, G, C). The split is necessary because the first overhang base has a 25 % probability of matching the genomic sequence by chance, which otherwise shifts the composite histogram and obscures the adaptor signal.

To reconstruct the adaptor consensus, the histogram with the highest total count is selected (say, the A histogram, implying the adaptor begins with A). The most frequent second base is then identified (say, G), and the G sub-histogram, shifted by one position, is added to the A histogram. The resulting composite histogram is extremely clean: on real datasets, base prevalence at each of the first ∼40 adaptor positions typically exceeds 70 % (Figure 1).

9.2 Pilot blocks and reprocessing

Adaptor inference is part of the broader pilot-block mechanism described in Section 4. Once the adaptor consensus is called, any pilot block that was processed without it is reprocessed from scratch; no reads are lost.

9.3 Trimming and alignment extension

Once an adaptor is known, each read's overhang is compared to it. If enough bases match, the alignment is clipped back to the exact genomic position where the adaptor begins, effectively extending the reported alignment. In the Magic2 output format the trimmed bases are exported as clipped, and the extension bases are reported in upper case rather than lower case. A read that aligns fully up to the recognised adaptor boundary, with no mismatches in the genomic portion, is counted as a *perfect alignment*.

9.4 RNA-mode extras: poly(A) detection

RNA mode and strand bias are inferred during the pilot phase (Section 4). Once the strand is established, a poly(A) tail is sought at the 3′ end of forward reads and a poly(T) tail at the 5′ end of reverse reads, the two being equivalent by symmetry. Tails carrying at least eight consecutive A (or T) residues are recorded, accumulated across reads, and reported as candidate poly(A) addition sites.

9.5 Trans-splice leader detection (*C. elegans*)

In *C. elegans* the majority of mRNAs acquire a short, stereotyped sequence at their 5′ end by trans-splicing: the *splice leader* (SL). SL sequences are highly conserved and serve as extremely reliable markers of transcription start sites. Magic2 searches for SL sequences at the same positions used for adaptor detection: the 5′ end of forward reads and the 3′ end of reverse reads. The 12 known *C. elegans* SL sequences (SL1–SL12) are hard-coded in the program (**?**). Reads carrying a recognised SL are trimmed at the SL boundary, the SL identity is recorded, and the counts are reported in the TSF output, providing a direct census of trans-splicing activity across the transcriptome.

SL detection is activated by specifying the target organism in the run configuration. The recommended mechanism is a `Species=worm` entry in the `-T` target configuration file, which keeps species-specific settings alongside the genome index rather than on the command line. A `--species worm` command-line override is also accepted for convenience.

10 Output formats

10.1 Alignment: SAM and BAM

Magic2 writes alignments in SAM or BAM format (`wsa/sa.sam.c`). The SAM writer is self-contained: it produces a fully valid SAM file with no dependency on any external library. BAM output is requested via `--bam`: Magic2 checks whether `samtools view` is available in `PATH` and, if so, pipes its SAM output through `samtools view -bS` to produce a compressed BAM file. If `samtools` is not found, Magic2 falls back silently to uncompressed SAM and reports the fallback to the user. This keeps the binary free of any link-time dependency on `htslib` or `samtools`.

10.2 Compact hits format

The `.hits` format (`wsa/sa.tabular.c`), selected via `--hits`, is a compact tab-separated alignment format designed for internal Magic2 pipelines where BAM compatibility is not required. It is faster to write and parse than SAM and represents multi-exon alignments more compactly.

10.3 TSF: Tag–Sample Format

All scalar outputs — expression counts, intron support tables, poly(A) sites, QC metrics, adaptor profiles — are exported in TSF (Tag–Sample Format): a plain tab-separated file with columns `run` / `tag` / `format-code` / `value`, where format codes are `i` (integer), `f` (float), and `t` (text). TSF files need not be sorted; multiple TSF files can be concatenated directly to merge runs; and the `tsf.c` library can pivot any TSF file into a two-dimensional table compatible with Excel.

10.4 Coverage tracks (wiggle / BigWig)

Coverage profiles are exported strand-specifically by `wsa/sa.wiggle.c` in the UCSC *fixedStep* wiggle format (`.BF.gz`), which is the most compact text wiggle encoding. Each chromosome block opens with a header line of the form

```
fixedStep chrom=chr1 start=1 step=10
```

followed by one coverage value per line per step position. The step size is chosen automatically from genome length: 1 for small genomes (bacteria, yeast) and 10 for large genomes (human, mouse), consistent with Magic2's general policy of deriving all internal parameters from the data rather than requiring user input. The step can be overridden with `--wiggle_step N` when a specific resolution is needed. Read-start and read-end profiles are also available for transcription-start-site (TSS) and transcription-end-site (TES) analysis.

The current standard for genome browsers (UCSC, IGV, Ensembl) is BigWig: an indexed binary format. BigWig cannot be produced natively without linking the UCSC Kent library, which Magic2 deliberately avoids. The recommended workflow is therefore identical to the BAM strategy: Magic2 checks whether `wigToBigWig` is available in `PATH` and, if so, pipes the fixedStep wiggle output through it, also writing the required chromosome-sizes file from the genome index. If the tool is absent, the compressed fixedStep file is kept. `wigToBigWig` is distributed as a standalone static binary by UCSC (`https://hgdownload.cse.ucsc.edu/admin/exe/`) and is also installable via `conda install ucsc-wigtobigwig`.

[TODO: implement the `wigToBigWig` pipe and chromosome-sizes export in `wsa/sa.wiggle.c`.]

10.5 Gene expression: projecting coverage onto the annotation

The annotation supplied as class `A` serves two distinct purposes. The first is the construction of junction seeds (Section 5.7), which happens at index time. The second is gene expression, which happens at output time and

is described here.

Why projection rather than read counting. Conventional pipelines obtain gene counts in a second pass: a tool such as `htseq-count` or `featureCounts` re-reads the BAM file, tests each alignment against the annotation, and assigns or discards it. Magic2 does not need that pass, because it has already built the object that the pass would reconstruct. A strand-specific coverage map at single-base resolution is accumulated during alignment, while the reads are in memory, at no additional I/O cost; it is required in any case for the coverage tracks (Section 10.4).

That map is a sufficient statistic for expression. Once per-base, per-strand depth is known, the expression of any feature is an integral of the map over the geometry of that feature. Counting therefore ceases to be an operation on reads and becomes an operation on the annotation, and costs no further traversal of the data. (A new annotation can in principle be re-projected onto an exported coverage track without realigning, but the point is academic: realigning is cheap enough that nobody would bother.) Note that the internal map is kept at single-base resolution regardless of `wiggle_step`, which only governs the resolution of the exported track; gene counts are therefore exact even when coverage is exported at 10-base resolution.

Slicing the genome by annotation status. The annotation is projected onto the genome, and the genome is cut into maximal intervals of constant annotation status. Each interval is classified as intergenic, intronic, exonic sense, exonic antisense, or CDS, and carries the list of genes claiming it. This partition is a static property of the annotation: it is computed once, independently of any data.

The reformulation is what makes overlapping genes tractable. The awkward question "which gene does this read belong to" — which must be asked once per read, and has no answer for reads in shared regions — is replaced by "which genes claim this interval", which is asked once per interval and always has an answer.

Resolving ambiguity. An interval claimed by a single gene contributes its coverage integral to that gene directly. The difficult case is an interval claimed by two genes, classically a convergent pair sharing a $3'$ UTR. In a stranded library the strand resolves it. In an unstranded library nothing in the data can.

Magic2 defers such intervals rather than deciding them. Once every unambiguous interval has been attributed, the deferred signal is divided between the claimants in proportion to the coverage each has already accumulated over the intervals it claims alone. Each gene is thus weighted by evidence untouched by the ambiguity being resolved. The division is performed once and is not iterated to convergence. The two obvious alternatives are both biased: discarding ambiguous intervals penalises precisely those genes that overlap others, while crediting both claimants inflates both.

Quality control for free. The same partition yields the mapping-class fractions reported in `runStats.tsf` (Section 10.9) at no extra cost, and they are diagnostic. A high intronic fraction indicates pre-mRNA or nuclear RNA rather than mature message; a high intergenic fraction indicates genomic DNA contamination or transcription the annotation does not know about; the balance between exonic sense and exonic antisense measures the strand-specificity of the protocol directly.

The expression index. Raw fractional counts are not comparable across libraries. When one gene captures 40 % of the reads — haemoglobin in whole blood, or residual rRNA — every other gene's share of the total is compressed by that factor, and a comparison between two runs with different high-abundance content measures their high-abundance content rather than their biology. Magic2 therefore computes a preliminary expression index in which the contribution of super-expressed genes, defined as those capturing more than 1 % of all counts, is removed from the normalising denominator. Those genes are retained in the table with their own counts intact; only their weight in the total is withdrawn, which is what keeps the index of a housekeeping

gene stable across libraries of very different composition. Counts are exported in TSF (Section 10.3) and merge across runs by concatenation, so that a differential-expression table over an arbitrary number of runs requires no additional tooling.

Chromosome naming. Projection presupposes that the coordinates of the annotation and of the genome refer to the same objects. Magic2 does not attempt to reconcile naming conventions: chromosome identifiers are taken verbatim from the `G` FASTA and the `A` annotation, and a mismatch is a fatal error at index construction rather than a silent loss of every gene on the offending chromosome.

10.6 Intron support tables

Intron evidence is exported in two TSF files per run.

Single-intron table (`introns.tsf`). Each record identifies one intron and one run. The tag field encodes the intron as `chrom__donor_acceptor`, where the double underscore separates the chromosome name from the two boundary coordinates and a single underscore separates donor from acceptor. Both coordinates are positive; the strand is encoded by their order rather than by their sign. An ascending pair ($a_1 < a_2$) denotes an intron on the minus strand, a descending pair ($a_1 > a_2$) an intron on the plus strand. The format code is `iit` (two integers, one text): `n` (number of supporting reads on the annotated strand), `nR` (reads on the opposite strand), and `feet` (splice signal, e.g. `gt_ag`, `ct_ac`, `gc_ag`). A typical record is:

```
chr1__146546_138479 A_Total_151PE iit 4 0 gt_ag
```

The chromosome name is taken verbatim from the genome FASTA file and the optional GTF annotation; it appears unchanged in all output files, so coordinates are unambiguous across tools.

Double-intron table (`double_introns.tsf`). Each record reports a pair of introns observed within the same read. This is more informative than single-intron support alone: a read spanning two consecutive introns confirms the intervening exon as a genuine transcribed unit, providing direct exon-connectivity evidence for transcript reconstruction. The tag encodes both introns as `chrom__a1_a2___b1_b2` (triple underscore between the two intron coordinate pairs). The format code is `iitt`: `n`, `nR`, `feet1`, `feet2`.

Both files are directly convertible to Excel-compatible tables via:

```
tsf -i introns.tsf -I tsf -O table -o introns_table.txt
```

and can be merged across runs by simple concatenation, since TSF files require no sorting.

10.7 Poly(A) addition sites

Candidate poly(A) addition sites are exported in `polyA.tsf`. Each record identifies one site, one run, and one strand. The tag field has the form `G.chrom__position`, where `G` denotes a genomic locus (as opposed to a gene model coordinate), the chromosome name is verbatim from the FASTA, and the position is the last aligned base before the poly(A) tail. The format code is `ti` (one text, one integer): `Strand` (`+` or `−`) and `n` (number of reads with a poly(A) or poly(T) tail ending at this position). A typical record is:

```
G.chr1__1304288 A_Total_151PE ti + 5
```

Sites supported by only one read are retained in the file but should be treated with caution; reliable poly(A) sites typically accumulate tens to hundreds of reads at a single nucleotide position, reflecting the precision of cleavage-and-polyadenylation.

10.8 Error profile

The per-run substitution and indel profile is exported in `errorProfile.tsf`. The format code is `i` throughout; the tag encodes the error type directly. The 12 substitution types are named `X>Y` (reference base to observed base, e.g. `A>G`); diagonal entries (`A>A` etc.) are always zero by construction and serve as a consistency check. Insertions and deletions of one base are counted separately by the inserted or deleted base (`InsA`, `DelT`, etc.). `Any` gives the total error count across all categories. A typical file:

```
A_Total_151PE Any i 4880463
A_Total_151PE A>G i 299012
A_Total_151PE G>T i 581599
A_Total_151PE InsA i 73057
A_Total_151PE DelA i 52330
```

This profile characterises the sequencing technology rather than the biology: the balance between transitions (`A>G`, `G>A`, `C>T`, `T>C`) and transversions (`A>C`, `A>T`, `G>C`, `G>T`, `C>A`, `C>G`, `T>A`, `T>G`) in the mismatch table reflects the error chemistry of the sequencer, not the underlying SNP or mutation spectrum. The profile is useful for comparing sequencing quality across runs, for detecting systematic chemistry artefacts, and for tuning the `errCost` parameter for non-Illumina technologies.

10.9 Run statistics

The file `runStats.tsf` collects all global alignment statistics for a run in a single TSF file (`wsa/sa.stats.c`). The header comment records the index used, seed length, maximum repeats, and error cost, providing a self-contained audit trail.

`runStats.tsf` uses multi-value format codes (`ift`, `iftift`, `itititft`, ...) that pack several related numbers and labels into one record. This keeps the file human-readable as a flat list while allowing the `tsf` tool to expand it into a table. Tags with zero counts are sometimes omitted.

The main categories are as follows.

Pair and read counts. `Pairs`, `Aligned_pairs`, `Compatible_pairs` (correct orientation and insert size), `Circle_pairs` (both mates on the same strand — a library artefact), and `Non_compatible_pairs`. At read level: `Aligned_reads`, `Unaligned_reads`, `Perfect_reads` (zero mismatches, zero indels), `Partial_reads` (one mate aligned), and `Complex_reads` (unusual alignment structure). `Low_entropy_reads` counts reads filtered out before alignment due to insufficient sequence complexity.

Base counts. `Bases` gives total, R1, and R2 base counts as three integers (`iii`). `Aligned_bases` gives total, R1, and R2 aligned counts each with their percentage of raw bases (`ififift`).

Error summary. `Errors` gives the total error count and its percentage of aligned bases. `Mismatches_and_InDels` gives the subset of errors that are substitutions or short indels (1–3 bases), excluding intron-spanning gaps. The full per-type breakdown is in `errorProfile.tsf` (Section 10).

Adaptor and clipping statistics. `Clipped_3prime_Adaptor_read1` records the number of reads trimmed and the inferred adaptor sequence. `Clipped_polyA` and `Clipped_polyT` count reads trimmed at a poly(A) or poly(T) tail.

Intron statistics. `Intron_supports` gives the total number of intron-spanning reads, broken down by splice signal (GT-AG, CT-AC, other) and the fraction on the positive strand. `Supported_introns` gives the number of distinct intron loci passing the reliability threshold (GT-AG seen once, or any signal seen at least three times).

Multi-mapping distribution. `Reads_Aligned_once` through `Reads_multi_aligned__10` give the number and percentage of reads aligning to exactly 1, 2, ..., 10 loci. `Reads_aligned_several_times` is the total fraction with more than one alignment.

Target-class breakdown. Magic2 aligns against one or more named target classes defined in the index (Section 5). The classes are those declared in the target configuration (Section 5.1): most commonly `G` (genomic, the primary genome), `M` (mitochondrial), `R` (ribosomal RNA) and `S` (interspersed repeats). For each class, `Reads_aligned_in_class_X` and `Bases_aligned_in_class_X` give total counts, positive- and negative-strand breakdowns, the positive-strand fraction, and the percentage of all aligned reads or bases. The strand balance within each class is informative: in a stranded RNA-seq library, class G reads show roughly 50/50 strand split across the whole genome but strong strand bias within individual genes, while class R reads are nearly all on one strand because ribosomal RNA transcription is unidirectional.

11 Automatic parameter selection

Magic2 derives all internal parameters from the hardware and data rather than requiring user configuration. Hardware auto-configuration (NUMA node selection, CPU count, agent and block counts) is described in Section 3. Data auto-configuration (read type, library type, strand, adaptor sequences) is described in Section 4. The resulting defaults are listed below for reference; all can be overridden on the command line when needed.

Parameter	Default
`errCost`	4 or 8, selected dynamically from the pilot data
`errRateMax`	12 %
`bMax`	3 Mb per block
Index step	6 (genomes $>$ 1 Mb)
Alignment step (`iStep`)	`tStep`
`seedLength`	16 (genomes $<$ 1 Gb); 18 (human/mouse)
`maxTargetRepeats`	81 (Section 5.6)
`wiggle_step`	1 (small genomes); 10 (human/mouse)
`minAli`	30
`minScore`	30
`maxIntron`	1,000,000 bases

12 Build system and supported platforms

Magic2 is written in C and built with `make` and `tcsh`. The repository lives inside the global acedb distribution ($\sim$700,000 lines of C); Magic2 depends on acedb libraries (notably `w1`, `wego`) and has not yet been fully separated.

Setting	Value
Optimised Linux build flag	`ACEDB_MACHINE=LINUX_4_OPT`
64-bit variant	`ACEDB_MACHINE=LINUX_X86_64_4_OPT`
Output binary	`Magic2/bin.LINUX_4_OPT/magic2`
Primary platform	Linux x86-64
Secondary platforms	Linux ARM, macOS
Optional GPU acceleration	CUDA (Thrust); silent CPU fallback

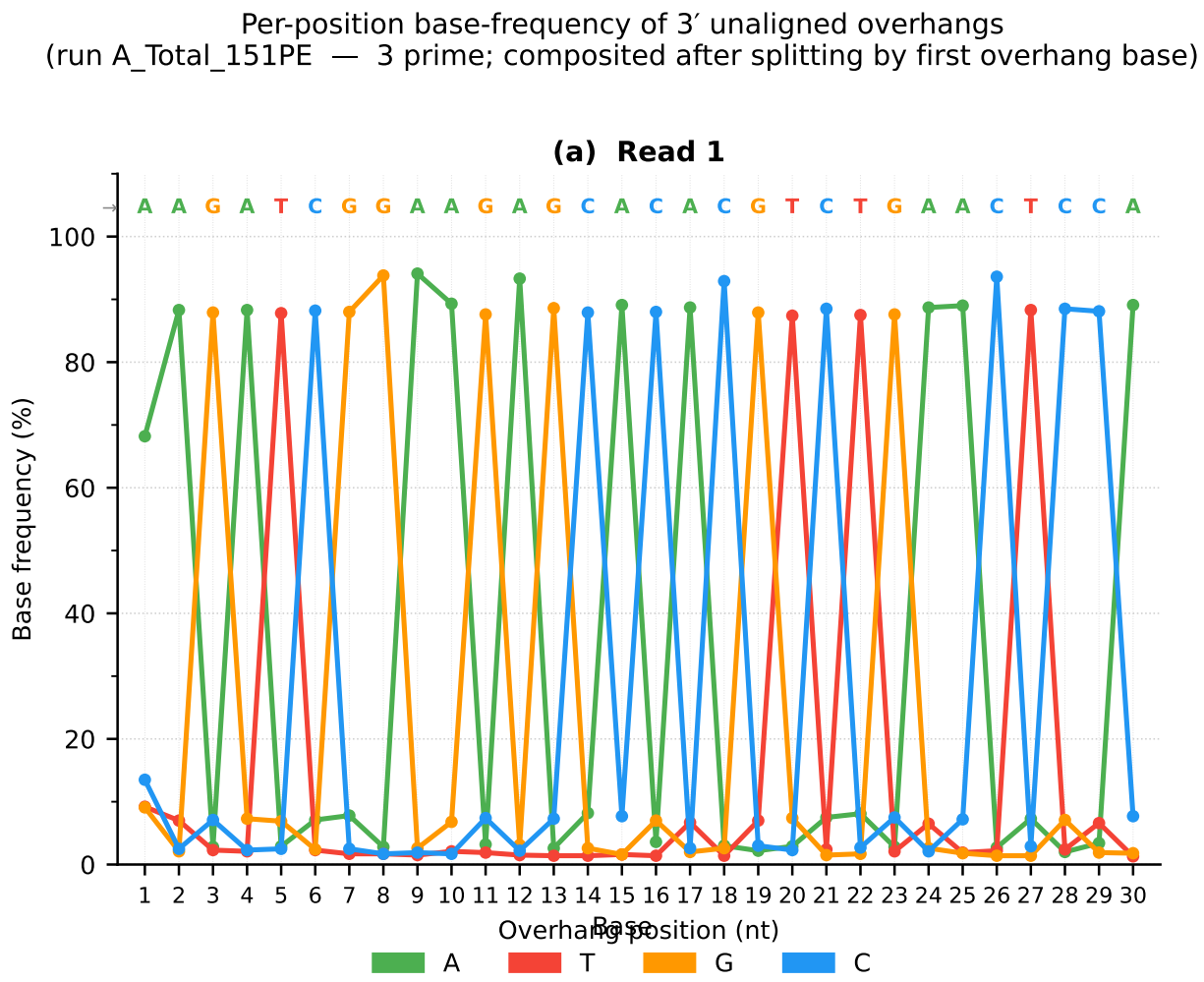

Figure 1: Per-position base-frequency of 3′ unaligned overhangs (run A_Total_151PE), after splitting by first overhang base and compositing. **(a)** Read 1 (TruSeq adaptor): the four bases alternate sharply between positions, each dominant base reaching 88–94 %, unambiguously encoding the 29-nt consensus shown above the panel. **(b)** Read 2 (degenerate adaptor): dominant-base fractions range from 50–80 %, consistent with a partially randomised ligation sequence; the adaptor family is nonetheless uniquely identified. Colours: A = green, T = red, G = orange, C = blue. The coloured sequence above each panel is the adaptor inferred by Magic2.